

Alba: A Multimodal Rendering System for Autonomous Vehicle Simulation

Ryusuke Villemin
rvillemin@aurora.tech
Aurora Innovation
USA

Anton Gribovskiy
agribovskiy@aurora.tech
Aurora Innovation
USA

Magnus Wrenninge
mwrenninge@aurora.tech
Aurora Innovation
USA

I-Chen Jwo
ijwo@aurora.tech
Aurora Innovation
USA

Steve Capell
scapell@aurora.tech
Aurora Innovation
USA

Steve Bako
sbako@aurora.tech
Aurora Innovation
USA

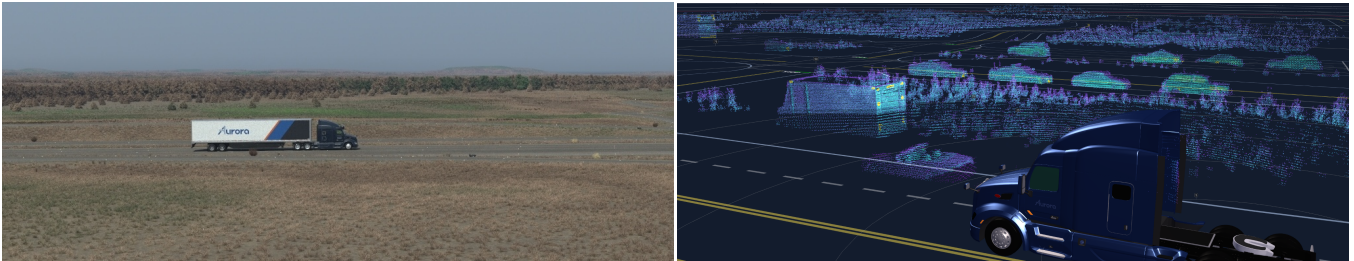


Figure 1: Examples of sensor simulations created by Alba. Left: a witness camera of the autonomous truck on a Texas highway. Right: Visualization of lidar data as seen by the autonomy system.

ABSTRACT

Physically based rendering and path tracing have become the norm in visual effects and animation thanks to the level of realism that they offer. On top of these foundations, modern rendering systems implement layers of specializations that target the use case of images for human consumption. In this paper, we discuss Alba, a multimodal rendering framework for camera, lidar and radar, which targets the use case of autonomous vehicle simulation. We discuss its architecture and the different design choices made to optimize for accuracy, scale, and machine consumption.

ACM Reference Format:

Ryusuke Villemin, Magnus Wrenninge, Steve Capell, Anton Gribovskiy, I-Chen Jwo, and Steve Bako. 2024. Alba: A Multimodal Rendering System for Autonomous Vehicle Simulation. In *Special Interest Group on Computer Graphics and Interactive Techniques Conference Talks (SIGGRAPH Talks '24)*, July 27–August 01, 2024, Denver, CO, USA. ACM, New York, NY, USA, 2 pages. <https://doi.org/10.1145/3641233.3664330>

1 INTRODUCTION

While the main goal of the introduction of physically based rendering [M. Pharr and Humphreys 2016] and ray-tracing [Whitted 1979] has been to create images indistinguishable from reality, the

requirements for what makes an image sufficiently realistic depends on the client. In games, there is an additional constraint on the FPS that needs to be maintained. In the autonomous vehicle (AV) space, the client is not a human being but a machine learning model that needs to be trained or tested, and to that end, the gap between reality and simulation needs to be small enough that performance on simulated scenes accurately represents real-world performance. Additionally, because of the amount of testing data needed in such contexts, the system needs to produce orders of magnitudes more data than a typical visual effects production. In this work, we present Alba, a sensor simulation framework encompassing camera, lidar and radar light transport simulation as well as DSP and other device-specific processing.

2 SYSTEM OVERVIEW

Alba is an end-to-end rendering system that is tailored to creating synthetic sensor data for autonomous vehicle training and testing. Input scenes are created in Houdini [SideFX 1996] and exported as OpenUSD [Pixar 2016] files and include everything from the renderable geometry to ML labeling ground truth and sensor configurations.

In order to efficiently produce concurrent output streams across all modalities, Alba diverges from the common frame-by-frame approach used for film production and instead maintains the ray tracing scene structure over a time interval that covers multiple firings of each sensor instance. Through this process, Alba provides the time-varying geometry representation needed for camera motion blur, lidar sweeps and radar doppler effects. Additionally, this architecture allows Alba to parallelize execution of almost twenty

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
SIGGRAPH Talks '24, July 27–August 01, 2024, Denver, CO, USA
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0515-1/24/07
<https://doi.org/10.1145/3641233.3664330>

cameras and around ten lidars and radars against the same acceleration structures, ensuring that even very large multi-core machines can be utilized effectively, even when there are large differences between the fastest and slowest computations. The parallelization and optimization also extends into each sensor modality, with finer-grain multi-threading and vectorization at several levels.

Finally, as the majority of our sensor simulations are used to test the perception subsystem in isolation (and do not require end-to-end simulation), the system also offers a temporal parallelization option, which computes different simulation time intervals on separate machines and later fuses the results, allowing for fast turn-around.

3 CAMERA SIMULATION

Alba's camera simulator is a radiometrically calibrated spectral path tracer working within the visible spectrum band (380 to 780 nm). However, rather than directly outputting the images generated by the path tracer, our integrator instead computes the radiance that strikes the photocells on the camera sensor, allowing physically based simulation not just of the light transport, but also of the effect of the color filter array (CFA), sensor characteristics, and the image signal processor (ISP) which converts raw sensor readings into an RGB image stream.

In order to manage Monte Carlo (MC) integration noise, we utilize a retrained version of Intel's Open Image Denoise [Intel 2020], which improves performance on images with spectral color noise. Particular care is taken to denoise before the simulation of CFA and sensor characteristics, while also accounting for the camera lens point spread function (PSF).

Alba also diverges from typical production renderers in its shading system. Because the goal is physical realism rather than art directability, we utilize a monolithic material model rather than shading networks. The material model provides a unified set of layered BRDFs which cover not only visible light but also near-infrared and GHz radar frequencies, ensuring that all objects provide the appropriate response in all three modalities.

4 LIDAR SIMULATION

Our lidar renderer is also based on Monte Carlo integration and handles both time-of-flight (ToF) and frequency-modulated continuous-wave (FMCW) simulation in a common framework. In both cases, the simulator considers the spatial character of the laser beam (e.g., waist, spread, cross section shape), Doppler effects, as well as near-IR material properties on objects that wholly or partially intersect the beam. Each MC sample deposits its estimated radiance in a temporal (for ToF) or frequency (for FMCW) histogram, from which peak-finding algorithms extract one or more returns, closely mimicking the processes used in the physical devices. Additionally, we utilize measured noise models to emulate probability of detection and range uncertainty, which are crucial to ensuring accurate behavior for measurements at the limits of what the physical devices can detect.

We note the importance of simulating firing times accurately for each lidar beam, as these serve as important feature cues for ML models that look at multiple sweeps of lidar jointly in order to

estimate object velocity. Modern ray tracing APIs that consider ray time and multi-sample motion blur are key to accurate simulations.

5 RADAR SIMULATION

Monte Carlo integration also forms the foundation of our radar simulator, which is used to model multiple types of MIMO-based FMCW imaging radar devices [Wikipedia 2024]. Because radar has a wide field of view and small but highly-reflective objects can produce strong returns, our system employs spatial importance sampling to minimize variance. Each MC sample accumulates radiance in a 4D data cube (covering azimuth, elevation, range and range-rate), from which returns are extracted using peak finding, similar on the physical devices. The simulator also factors in device specifics such as noise, losses, coherent gain as well as range ambiguity.

6 CONCLUSIONS

Alba's design and architecture enables Aurora to generate the large amounts of synthetic sensor data needed to test and validate our autonomy systems. The system has been in use since 2021 and its accuracy is evaluated in our safety case framework [Aurora 2022].

ACKNOWLEDGMENTS

The authors would like to recognize Wenqing Wang and Alex Glavin, who made important contributions to the system.

REFERENCES

- Aurora. 2022. Aurora's Safety Case Framework. <https://safetycaseframework.aurora.tech/gsn>.
- Intel. 2020. Open Image Denoise. <https://www.openimagedenoise.org/>.
- W. Jakob M. Pharr and G. Humphreys. 2016. *Physically Based Rendering*. Morgan Kaufmann.
- Pixar. 2016. USD. <https://openusd.org/>.
- SideFX. 1996. Houdini. <https://www.sidefx.com/products/houdini/>.
- Turner Whitted. 1979. An improved illumination model for shaded display. *Proceedings of the 6th annual conference on Computer graphics and interactive techniques* (1979). <https://api.semanticscholar.org/CorpusID:59809792>
- Wikipedia. 2024. MIMO radar. https://en.wikipedia.org/wiki/MIMO_radar.